

AccuKnox AI/ML Trainee Technical Assignment

NAME: Supritha M Y

Contact: 7676141929

Mail: suprithamyogendra@gmail.com

Problem 1 – API Data Retrieval and SQLite

This project retrieves book information from the Open Library API and stores the retrieved data in a local SQLite database.

Retrieved fields: Book title, author, and publication year.

The SQLite database contains a books table with an automatically generated ID, title, author, and publication year.

Result: Fantastic Mr Fox — Roald Dahl — 1970.

Implementation: Python, Requests, and SQLite.

Problem 2 – Data Processing and Visualization

This project retrieves student examination data from a public REST API, selects an examination, calculates the average score for each participant, and creates a bar chart.

Assumption: for the selected intensive_2_exam_3_2026 examination, each participant's average is calculated as the mean of their available problem scores.

Implementation: Python, Requests, and Matplotlib.

Output: student_average_scores.png, containing the bar chart of student average scores.

Problem 3 – CSV Data Import to SQLite

This project reads user information containing names and email addresses from a CSV file and inserts the records into a local SQLite database.

The users table contains an automatically generated ID, name, and email. Duplicate email records are ignored using a unique constraint.

Implementation: Python csv module and SQLite.

GitHub repository: <https://github.com/Supritha-m-y/accuknox-ai-ml-assignment>

4. Link to my most complex personally written Python code

Heart Disease Prediction Using Machine Learning – MSc Data Science Project, Teesside University

This project involved data preprocessing, exploratory data analysis, visualisation, machine learning model development, cross-validation, hyperparameter tuning, and model evaluation.

Code: https://github.com/Supritha-m-y/heart-disease-prediction-ml/blob/main/Heart_Disease.ipynb?utm_source=chatgpt.com

5. Link to my most complex personally written database code:

API Data Retrieval and SQLite Database – AccuKnox AI/ML Trainee Assignment

GitHub: <https://github.com/Supritha-m-y/accuknox-ai-ml-assignment/blob/main/problem-1-api-sqlite/problem1.py>

Assignment 2

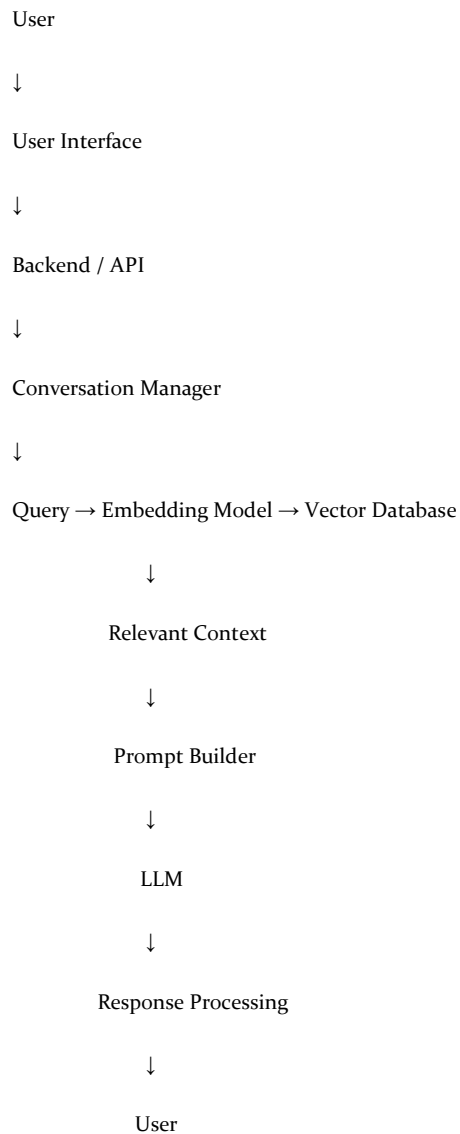
1. Self-Rating

LLM	B
DEEP LEARNING	B
AI	A
MACHINE LEARNING	A

I would rate myself **B in LLM and Deep Learning, and A in AI and Machine Learning**. I have a strong understanding of AI and machine learning concepts and practical exposure through my academic work and projects. I am comfortable with machine learning workflows, model development, evaluation techniques, and applying appropriate approaches to data-driven problems.

For LLMs and Deep Learning, I have a good foundation in the key concepts and practical applications, including neural networks, embeddings, transformers, prompt construction, and retrieval-augmented generation. I am continuing to strengthen my hands-on expertise in these areas through further implementation and experimentation.

2. High-Level Architecture of an LLM Chatbot



An LLM chatbot typically consists of the following key components:

1. User Interface

Provides the interface through which users submit questions and receive chatbot responses, such as a web or mobile application.

2. Backend / API Layer

Receives user requests, handles authentication and validation, manages application logic, and communicates with the remaining components.

3. Conversation Manager

Maintains relevant conversation history and context so the chatbot can generate responses that are consistent with previous interactions.

4. Knowledge Base

Contains domain-specific information such as documents, FAQs, manuals, policies, or other business data that the chatbot may need to answer user queries.

5. Embedding Model

Converts documents and user queries into numerical vector representations called embeddings, which capture their semantic meaning.

6. Vector Database / Retrieval Layer

Stores document embeddings and performs similarity searches to retrieve information that is semantically relevant to the user's query.

7. Prompt Builder

Combines the user's question, relevant conversation history, and retrieved information into a structured prompt for the LLM.

8. Large Language Model (LLM)

Processes the prompt and generates a natural-language response based on the provided context and its learned knowledge.

9. Response Processing

Validates, filters, formats, and returns the generated response to the user through the application.

3. Vector Database

A vector database is designed to store and search numerical vector representations of data, commonly called embeddings. Embeddings allow information to be compared based on semantic similarity.

For example, a company chatbot could convert internal documents into embeddings and store them in a vector database. A user's question can also be converted into an embedding, allowing the system to retrieve documents that are semantically relevant to the question.

Example Use Case

Consider a company chatbot that needs to answer questions about internal documents.

Company Documents



Text Extraction



Embedding Model



Vector Database



User Question



Question Embedding



Similarity Search



Relevant Documents



LLM



Answer

My Choice: FAISS

For a **small-scale prototype**, I would choose **FAISS (Facebook AI Similarity Search)**.

Reasons for choosing FAISS:

- It is open source.
- It provides efficient similarity search.

- It integrates well with Python.
- It is suitable for experimentation and prototypes.
- It can efficiently search large collections of vectors.
- It avoids the complexity of deploying a separate managed database for a simple prototype.

For a larger production application, I would evaluate alternatives such as a dedicated vector database based on requirements such as scalability, persistence, metadata filtering, distributed deployment, security, and operational requirements.

Conclusion

Vector databases are particularly useful in LLM applications because they allow the system to retrieve relevant information based on **semantic similarity**, rather than relying only on exact keyword matching. Combining vector search with an LLM is a common approach for building knowledge-based chatbots and retrieval-augmented generation systems.

Submission References GitHub repository: <https://github.com/Supritha-m-y/accuknox-ai-ml-assignment>

The repository contains the Python implementations and README files for Problems 1–3.